

SkyWatch — Protocol & Packet Reference

Wire format source-of-truth for every link in the system

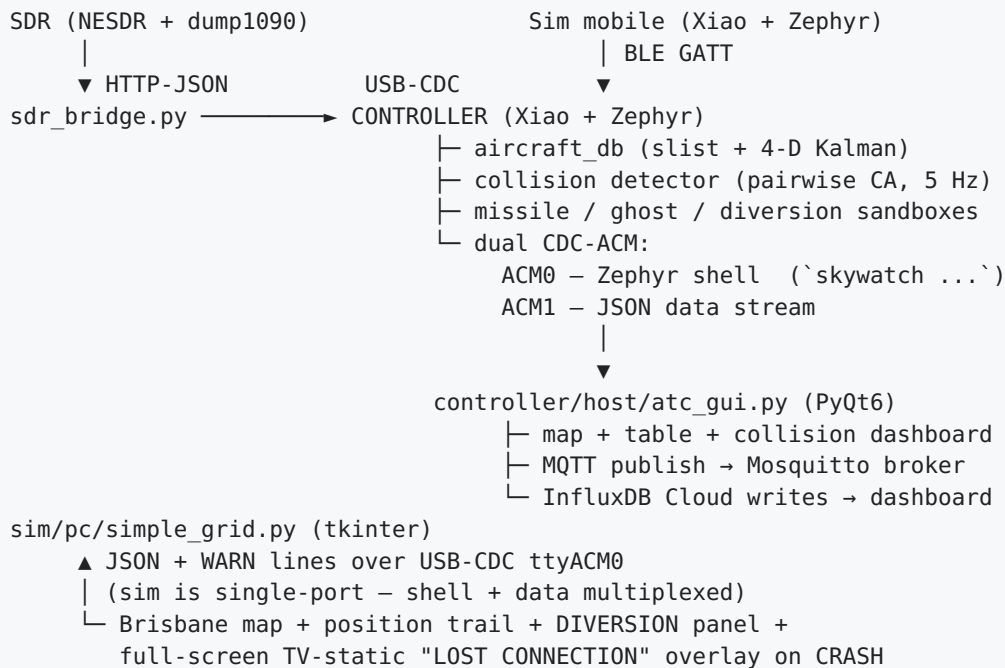
Vermillion-Puck — CSSE4011 final project · May 2026

Why this doc exists

When the diagrams in the wiki need a redraw (draw.io or otherwise), the person doing the drawing should not have to chase six different `.h` files to find which byte goes where. This document collects every wire format in the SkyWatch system — every struct, every JSON schema, every topic, every characteristic UUID — into one place. C structs are quoted **verbatim** from the source headers, not paraphrased.

If something here disagrees with the code, the code wins. The headers linked at each section are the actual source-of-truth.

System overview



The system has six distinct links. Each one is documented in its own section below, then summarised in a table at the very end.

1. BLE link — sim → controller (aircraft frame)

Property	Value
Direction	Sim mobile (peripheral) → Controller (central)
Transport	BLE GATT NOTIFY
Service UUID	4d8a4011-7f24-43c4-9c5b-a17c7af70001
Characteristic UUID	4d8a4011-7f24-43c4-9c5b-a17c7af70002
Advertising name	skywatch-sim
Frame size	38 bytes
Endianness	Little-endian (ARM Cortex-M native, zero conversion)
Update rate	5 Hz (200 ms)
Protocol version	1
ATT MTU	≥41 (mandatory MTU exchange after connect)

Source: `sim/mobile/src/common/src/ble_packet.h` (mirror at `controller/src/common/src/ble_packet.h`).

1.1 C struct (verbatim)

```
#define BLE_PROTOCOL_VERSION 1
#define BLE_FRAME_SIZE      38

enum ble_aircraft_source {
    BLE_SRC_ADS_B = 0, /* Only for completeness – sim normally sends BLE_SIM. */
    BLE_SRC_BLE_SIM = 1,
};

/* If a *_VALID bit is clear, the corresponding field's value is undefined
 * and MUST be ignored by the receiver. */
#define BLE_FLAG_ON_GROUND (1u << 0)
#define BLE_FLAG_ALT_VALID (1u << 1)
#define BLE_FLAG_VEL_VALID (1u << 2)
#define BLE_FLAG_HDG_VALID (1u << 3)

struct __packed ble_aircraft_frame {
    uint8_t source; /* enum ble_aircraft_source */
    uint8_t flags; /* BLE_FLAG_* */
    uint32_t icao24; /* low 24 bits; top byte = 0 */
    uint8_t _pad0[4]; /* zero-fill */
    double lat; /* WGS-84 degrees */
    double lon; /* WGS-84 degrees */
    int16_t alt_ft; /* valid only if flags & BLE_FLAG_ALT_VALID */
    int16_t vel_kt; /* valid only if flags & BLE_FLAG_VEL_VALID */
    int16_t hdg_deg; /* valid only if flags & BLE_FLAG_HDG_VALID, 0..359 */
    uint8_t reserved; /* 0 */
    uint8_t _pad1[1]; /* alignment */
    uint32_t ts_ms; /* uint32 ms since sim-node boot */
};
```

1.2 Wire layout (38 bytes, little-endian)

Offset	Size	Field	Type	Notes
0	1	source	uint8	BLE_SRC_* enum
1	1	flags	uint8	BLE_FLAG_* bitfield
2	4	icao24	uint32 LE	24-bit ICAO in low 24 bits; top byte must be 0
6	4	padding	zero	aligns lat/lon to 8-byte boundary
10	8	lat	float64 LE	WGS-84 degrees [-90, 90]
18	8	lon	float64 LE	WGS-84 degrees [-180, 180]
26	2	alt_ft	int16 LE	feet MSL; ignore unless ALT_VALID
28	2	vel_kt	int16 LE	knots ground speed; ignore unless VEL_VALID
30	2	hdg_deg	int16 LE	degrees true 0..359; ignore unless HDG_VALID
32	1	reserved	uint8	0
33	1	padding	zero	alignment to next uint32
34	4	ts_ms	uint32 LE	ms since sim-node boot

1.3 Receiver-side validation

The controller's `ble_central.c` drops the notify on any of the following:

1. `notify length != BLE_FRAME_SIZE` (38).
2. `source ∉ {BLE_SRC_ADS_B, BLE_SRC_BLE_SIM}`.
3. `icao24 > 0xFFFFF` (top byte of the uint32 is nonzero).
4. `lat ∉ [-90, 90]` or `lon ∉ [-180, 180]`.
5. If a `*_VALID` flag is clear, the corresponding numeric field is treated as **absent** (not stored in `aircraft_db`).

2. BLE link — controller → sim (warning frame)

Property	Value
Direction	Controller (central) → Sim mobile (peripheral)
Transport	BLE GATT WRITE (no response)
Service UUID	ab340001-1234-5678-abcd-1234567890ab
Characteristic UUID	ab340002-1234-5678-abcd-1234567890ab
Frame size	16 bytes
Integrity	XOR-CRC over the first 15 bytes (last byte)
Cadence	On level transition + re-fired every ~8 s while WARNING-active

Source: `controller/src/common/src/ble_packet.h`.

2.1 C struct (verbatim)

```
#define BLE_WARNING_FRAME_SIZE 16

enum ble_warning_msg_type {
    BLE_WARN_MSG_COLLISION = 0x02,
    BLE_WARN_MSG_DIVERSION = 0x03,
    BLE_WARN_MSG_FREETEXT  = 0x04,
    BLE_WARN_MSG_CRASH     = 0x05,    /* terminal "you crashed" event */
};

enum ble_warning_severity {
    BLE_WARN_SEV_ADVISORY = 0,
    BLE_WARN_SEV_WARNING  = 1,
    BLE_WARN_SEV_URGENT   = 2,
};

struct __packed ble_warning_frame {
    uint8_t  msg_type;    /* BLE_WARN_MSG_* */
    uint8_t  icao[3];     /* ICAO of conflicting (other) aircraft */
    uint8_t  lat[3];     /* unused for collision msg — zero-fill */
    uint8_t  lon[3];     /* unused for collision msg — zero-fill */
    uint16_t alt;        /* unused for collision msg — zero */
    uint8_t  severity;   /* BLE_WARN_SEV_* */
    int8_t   hdg_delta;  /* used by msg_type=0x03 only; zero here */
    uint8_t  timestamp;  /* low byte of k_uptime_get() */
    uint8_t  crc;        /* XOR of the preceding 15 bytes */
};

static inline uint8_t ble_warning_crc(const struct ble_warning_frame *f)
{
    const uint8_t *b = (const uint8_t *)f;
    uint8_t c = 0;
    for (int i = 0; i < BLE_WARNING_FRAME_SIZE - 1; i++) {
        c ^= b[i];
    }
    return c;
}
```

2.2 Wire layout

Offset	Size	Field	Type	Notes
0	1	msg_type	uint8	0x02 collision, 0x03 diversion, 0x04 freetext, 0x05 crash
1	3	icao[3]	uint8×3	ICAO of the threat (3-byte big-endian within the array)
4	3	lat[3]	uint8×3	unused for collision/crash — zero-fill
7	3	lon[3]	uint8×3	unused for collision/crash — zero-fill
10	2	alt	uint16	alt_delta field for diversion (CLIMB/DESCEND), else 0
12	1	severity	uint8	BLE_WARN_SEV_*
13	1	hdg_delta	int8	diversion direction — see semantics table below
14	1	timestamp	uint8	low byte of k_uptime_get() ms
15	1	crc	uint8	XOR of bytes 0..14

2.3 Per-msg_type semantics

msg_type	What the sim does
0x02 COLLISION	Red LED on, buzzer chirps for 5 s. Prints <code>WARN collision ICA0=<hex> sev=<0\ 1\ 2></code> to USB.
0x03 DIVERSION	Acts on the suggestion (heading turn or altitude change). Prints <code>WARN diversion ICA0=<hex> hdg_delta=<int> alt_delta=<int></code> .
0x04 FREETEXT	Operator-defined free text (up to 27 chars in a custom 28-byte frame variant — rarely used). Prints <code>WARN msg <text></code> .
0x05 CRASH	Freezes physics for 10 s, prints <code>WARN crash</code> , then auto-resets to St Lucia origin. PC GUI shows full-screen TV-static "LOST CONNECTION" overlay.

2.4 Diversion direction encoding (msg_type=0x03)

The controller's `pick_diversion()` encodes the chosen manoeuvre across both `hdg_delta` (the `int8` at byte 13) and `alt` (the `uint16` at byte 10):

Direction	hdg_delta	alt (re-purposed as alt_delta, signed)
LEFT	-30	0
RIGHT	+30	0
CLIMB	0	+1000 (ft)
DESCEND	0	-1000 (ft)
RTB (return)	+127 sentinel	0
HOLD (circle)	-127 sentinel	0

3. USB-CDC link — controller ↔ host (data port, ttyACM1)

Property	Value
Direction	Bidirectional on the same port
Transport	USB-CDC ACM (the controller's second CDC instance)
Framing	Newline-delimited JSON objects (one object per line)
Update rate	Egress: 5 Hz collision tick + per-aircraft on update. Ingress: 2 Hz (<code>sdr_bridge.py</code> polls <code>dump1090</code> at 2 Hz)
Schema source	<code>resources/standards/json_protocol.py</code> (Python TypedDict + validator)

Two frame types share this stream — distinguished by their `"type"` discriminator field:

- `{"type": "aircraft", ...}` — one observation of one aircraft.
- `{"type": "collision", ...}` — one pairwise approach assessment.

3.1 aircraft frame

Quoted verbatim from `resources/standards/json_protocol.py`:

```
class AircraftFrame(TypedDict, total=False):
    """One aircraft observation as it appears on the wire (JSON).

    Optional fields (alt, vel, hdg) are omitted from the JSON when not
    known – they are NEVER serialised as `null`. The firmware's Zephyr JSON
    parser rejects null-for-typed-field; absent keys are handled cleanly by
    the descriptor return bitmask.
    """
    # Required on every frame:
    type: Literal["aircraft"]
    icao: str          # 24-bit ICAO, lowercase 6 hex chars, no '~'
    lat: float        # WGS-84 degrees, -90..90
    lon: float        # WGS-84 degrees, -180..180
    ts: float         # POSIX epoch seconds, float (sub-second OK)
    source: Source    # provenance – fusion key on the controller
    # Optional (omit key entirely when unknown):
    alt: int          # feet, MSL (baro)
    vel: float        # ground speed in knots
    hdg: float        # track in degrees true, 0..360
    # Kalman 10 s-ahead projection – emitted once the controller's filter has
    # had >=3 updates for this aircraft. Always emitted together.
    pred_lat: float
    pred_lon: float

    _REQUIRED_KEYS = {"type", "icao", "lat", "lon", "ts", "source"}
    _OPTIONAL_KEYS = {"alt", "vel", "hdg", "pred_lat", "pred_lon"}
    _ALLOWED_SOURCES = {"ADS_B", "BLE_SIM", "FICT"}
```

Example wire frame (with optional alt/vel/hdg + Kalman projection):

```
{"type": "aircraft", "icao": "a1b2c3", "lat": -27.4698, "lon": 153.0251, "alt": 500, "vel": 120.5, "hdg": 270.0, "ts": 1580000000.0, "pred_lat": -27.4698, "pred_lon": 153.0251}
```

Validation constraints (enforced by `validate()` in `json_protocol.py`):

- `type == "aircraft"`.
- `icao`: 1..7 char hex string (no leading ~).
- `lat` ∈ [-90, 90], `lon` ∈ [-180, 180].
- Optional `alt` / `vel` / `hdg` / `pred_lat` / `pred_lon`: must be numeric **if present**; key is omitted (never `null`) when absent.
- `source` ∈ {`ADS_B`, `BLE_SIM`, `FICT`}.

3.2 collision frame

Emitted by `controller/src/collision/src/collision.c::emit_collision()`. Exact `snprintf` format string:

```
"{"type": "collision", "icao_a": "%s", "icao_b": "%s", "level": "%s", "tca_s": %.1f, "min_sep_m": %.0f, "actual_sep_m": %.0f, "ts": %.3f}"
```

where the three `%s` slots are conditional inline fragments:

- `, "actual_sep_m": %.0f` — always emitted (current Euclidean distance NOW).

- `,"alt_diff_ft":%d` — only when both aircraft have a reported altitude.
- `,"diversion":"%s"` — only when a diversion has been suggested for this encounter.

Required keys (validator):

```
_COLLISION_REQUIRED_KEYS = {"type", "icao_a", "icao_b",
                             "level", "tca_s", "min_sep_m", "ts"}
_ALLOWED_COLLISION_LEVELS = {"CLEAR", "ADVISORY", "WARNING", "CRASH"}
_ALLOWED_DIVERSIONS       = {"LEFT", "RIGHT", "CLIMB", "DESCEND",
                             "RTB", "HOLD"}
```

C enums quoted from `controller/src/collision/src/collision.h`:

```
enum collision_level {
    COLLISION_CLEAR      = 0,
    COLLISION_ADVISORY   = 1,
    COLLISION_WARNING    = 2,
    COLLISION_CRASH      = 3,
};

enum collision_diversion {
    DIVERSION_LEFT       = 0, /* turn -30° */
    DIVERSION_RIGHT      = 1, /* turn +30° */
    DIVERSION_CLIMB      = 2, /* +1000 ft */
    DIVERSION_DESCEND    = 3, /* -1000 ft */
    DIVERSION_RTb        = 4, /* head toward UQ St Lucia */
    DIVERSION_HOLD       = 5, /* enter holding circle */
};
```

Threshold #defines (same header):

```
#define COLLISION_ADVISORY_SEP_M 1000.0
#define COLLISION_ADVISORY_TCA_S 60.0
#define COLLISION_WARNING_SEP_M 500.0
#define COLLISION_WARNING_TCA_S 30.0
#define COLLISION_CRASH_SEP_M 100.0
#define COLLISION_CRASH_VERT_FT 328 /* ~100 m */
#define COLLISION_VERTICAL_SEP_FT 1000 /* gate – force CLEAR if vert > this */
#define COLLISION_TICK_MS 200 /* 5 Hz collision detector */
```

Example wire frame (WARNING-level encounter, both alt known, with diversion suggested):

```
{"type":"collision","icao_a":"a1b2c3","icao_b":"gh05t1","level":"WARNING","tca_s":23.5,"min_sep_m":250,"a
```

3.3 Ingress direction (host → controller)

`sdr_bridge.py` polls dump1090's `aircraft.json` HTTP endpoint at 2 Hz, normalises every record into an `AircraftFrame` (same schema as section 3.1, `source="ADS_B"`), and writes one JSON line per aircraft to the controller's data port. The controller's `json_parser.c` descriptor-parses each line into a `struct aircraft_t` and upserts it into `aircraft_db`.

Normalisation logic (verbatim from `json_protocol.py`):

```
def normalize_dump1090(ac: dict, ts: float) -> Optional[AircraftFrame]:
    if "lat" not in ac or "lon" not in ac:
        return None
    icao_raw = ac.get("hex", "") or ""
    icao = icao_raw.lstrip("~").lower()
    out: AircraftFrame = {
        "type": "aircraft",
        "icao": icao,
        "lat": float(ac["lat"]),
        "lon": float(ac["lon"]),
        "ts": ts,
        "source": "ADS_B",
    }
    alt = ac.get("altitude", ac.get("alt_baro"))
    if alt is not None:
        out["alt"] = alt
    vel = ac.get("speed", ac.get("gs"))
    if vel is not None:
        out["vel"] = vel
    hdg = ac.get("track")
    if hdg is not None:
        out["hdg"] = hdg
    return out
```

4. USB-CDC link — controller shell port (ttyACM0)

Property	Value
Direction	Bidirectional (Zephyr shell)
Transport	USB-CDC ACM (the controller's first CDC instance)
Framing	Newline-terminated text (<code>uart:~\$</code> prompt)
Log backend	<code>LOG_INF</code> / <code>LOG_WRN</code> / <code>LOG_ERR</code> from all firmware modules also goes here

4.1 skywatch subcommand vocabulary

Command	What it does
skywatch ping	Reply pong (liveness check)
skywatch ble stats	BLE connection state + scan / discovery / notify counters
skywatch db dump	List every aircraft currently in the DB
skywatch collision stats	Tick counter + pair count + advisory / warning counters
skywatch collision inject	Spawn two synthetic FICT aircraft converging head-on (no BLE needed)
skywatch collision stop	Halt the inject keep-alive
skywatch collision crash <icao_a> <icao_b>	Force-fire a CRASH event between two ICAOs already in the DB
skywatch collision ghost [tca_s] [miss_m] [alt_ft]	Spawn FICT aircraft on intercept with the BLE sim (default 23 s TCA, 0 m miss, alt-match)
skywatch collision ghost_at <icao> [tca_s] [miss_m] [alt_ft]	Same but target any ICAO (BLE / ADS-B / FICT)
skywatch collision ghost_stop	Cancel current ghost
skywatch missile launch [ttl_s] [turn_rate_dps] [speed_kt]	Launch a pure-pursuit missile from UQ St Lucia at the BLE sim (default 60 s / 15°/s / 500 kt)
skywatch missile cancel	Abort the in-flight missile
skywatch diversion <left\ right\ climb\ descend\ rtb\ hold>	Manually fire a diversion suggestion to the sim
skywatch kalman q <m/s ² >	Tune Kalman process noise (default 0.5)
skywatch kalman r <m>	Tune Kalman measurement noise (default 10)
skywatch kalman bench	1000 predict + update iterations, <5 ms target
skywatch usb stats	Data-port Rx counters (bytes, frames, ringbuf drops)

5. USB-CDC link — sim ↔ host (single port, ttyACM0)

After Will's commit `384ab27` the sim collapsed to a single CDC ACM instance. Position JSON, warning text lines, and the Zephyr shell all multiplex on the same port.

5.1 Sim → host — position JSON

Emitted at 5 Hz by `send_position_json()` in `sim/mobile/src/main/src/codein.c`. Exact `snprintf`:

```
snprintf(buf, sizeof(buf),
    "{\"lat\":%ld,\"lon\":%ld,\"alt\":%u,\"spd\":%u,\"hdg\":%u}\",
    (long)g_lat, (long)g_lon,
    (unsigned int)g_alt,
    (unsigned int)g_speed,
    (unsigned int)g_heading);
```

Schema:

Key	Type	Units	Notes
lat	int32	degrees × 1e7 (signed)	e.g. -274698000 = -27.4698°
lon	int32	degrees × 1e7 (signed)	e.g. 1530251000 = 153.0251°
alt	uint16	metres MSL	note the unit difference from BLE (feet)
spd	uint16	knots ground speed	
hdg	uint16	degrees true, 0..359	

Example wire frame:

```
{"lat": -274698000, "lon": 1530251000, "alt": 100, "spd": 50, "hdg": 270}
```

Units gotcha: altitude is metres on this link but feet on the BLE aircraft frame (section 1). The sim's `aircraft set` / `aircraft wasd qle` commands take metres; the firmware multiplies by 3.28084 before packing the BLE frame.

5.2 Sim → host — warning text lines

Triggered on incoming BLE warning frames (section 2). Each line is written via `usb_serial_println()` so the tkinter GUI (`simple_grid.py`) sees them on the same stream as the position JSON.

Format	Trigger
WARN collision ICA0=<6 hex> sev=<0 1 2>	msg_type=0x02 received
WARN diversion ICA0=<6 hex> hdg_delta=<int> alt_delta=<int>	msg_type=0x03 received
WARN msg <text up to 27 chars>	msg_type=0x04 received (rare)
WARN crash	msg_type=0x05 received — triggers 10 s freeze + reset
WARN clear	Auto-fired ~5 s after the last warning, clears the GUI panel

5.3 Host → sim — shell commands (`aircraft ...`)

Sent as newline-terminated text into the same port. Zephyr's shell dispatches.

Command	What it does
<code>aircraft status</code>	Print current position JSON
<code>aircraft set <lat_e7> <lon_e7> <alt_m> <kt> <hdg></code>	Set exact state (lat/lon × 1e7, alt in metres, knots, degrees)
<code>aircraft wasd w</code>	+10 kt
<code>aircraft wasd s</code>	−10 kt
<code>aircraft wasd a</code>	−10° (left turn)
<code>aircraft wasd d</code>	+10° (right turn)
<code>aircraft wasd q</code>	+50 m altitude
<code>aircraft wasd e</code>	−50 m altitude
<code>aircraft reset</code>	Snap back to St Lucia origin (-27.4975°, 153.0137°, 100 m, 0 kt, 0°). Also clears warnings.
<code>aircraft circle <kt> [radius_m]</code>	Auto-fly circles (radius default 500 m). <code>aircraft circle 0</code> to stop.

6. MQTT — controller host → Mosquitto broker

Property	Value
Direction	<code>controller/host/atc_gui.py</code> → Mosquitto broker (local)
Transport	TCP, MQTT 3.1.1
Broker	<code>127.0.0.1:1883</code> (configurable via env)
QoS	0 (fire-and-forget)
Retention	True (latest payload sticks on every topic)
Keepalive	30 s
Client ID	<code>skywatch-atc-<hostname></code>
Last-will	<code>skywatch/status</code> ⇒ "offline", retained

Source: `controller/host/mqtt_publisher.py`.

6.1 Topic tree

Topic	Payload	Notes
<code>skywatch/collision</code>	full collision JSON (same as section 3.2)	every collision frame the controller emits
<code>skywatch/collision/<icao_a>_<icao_b></code>	same JSON	per-pair, ICAOs lexicographically sorted
<code>skywatch/status</code>	text "online" / "offline"	online on connect; offline via LWT

ICAO ordering is enforced by the publisher — the topic uses `min(a, b) + "_" + max(a, b)` so subscribers can pre-compute the expected per-pair topic without worrying which direction the frame arrived in.

6.2 Publish logic (verbatim excerpt)

```
TOPIC_BASE      = "skywatch"
TOPIC_COLLISION = f"{TOPIC_BASE}/collision"
TOPIC_STATUS    = f"{TOPIC_BASE}/status"
DEFAULT_HOST    = "127.0.0.1"
DEFAULT_PORT    = 1883

def publish_collision(self, frame: dict) -> None:
    payload = json.dumps(frame, separators=(",", ":"))
    a       = frame.get("icao_a", "?")
    b       = frame.get("icao_b", "?")
    per_pair = f"{TOPIC_COLLISION}/{a}_{b}"
    self._enqueue(TOPIC_COLLISION, payload, qos=0, retain=True)
    self._enqueue(per_pair,          payload, qos=0, retain=True)
```

7. InfluxDB Cloud — controller host → InfluxDB

Property	Value
Direction	controller/host/atc_gui.py → InfluxDB Cloud
Transport	HTTPS REST (via influxdb-client Python SDK)
Org	Blaise&Will.Incorporated (configurable via INFLUXDB_ORG)
Bucket	skywatch (configurable via INFLUXDB_BUCKET)
Token	env var INFLUXDB_TOKEN (set by controller/host/run_skywatch.sh)
Heartbeat	10 s tick — keeps the dashboard fresh between collision events

Source: controller/host/influx_writer.py .

7.1 Measurement `skywatch_events`

One Point per collision frame.

Tags (indexed — use these in `GROUP BY` / `WHERE`):

Tag	Value
<code>pair</code>	<code><icao_a>_<icao_b></code> lexicographically sorted
<code>level</code>	<code>CLEAR</code> / <code>ADVISORY</code> / <code>WARNING</code> / <code>CRASH</code>
<code>icao_a</code>	first ICAO
<code>icao_b</code>	second ICAO

Fields (numeric, unindexed):

Field	Type	Notes
tca_s	float	seconds to closest approach (0.0 if diverging)
min_sep_m	float	projected minimum separation, metres
controller_ts	float	controller uptime (seconds) from the wire frame
alt_diff_ft	int	alt_a - alt_b — written only when both aircraft reported altitude

Writer excerpt (verbatim):

```
p = (Point("skywatch_events")
    .tag("pair", pair)
    .tag("level", level)
    .tag("icao_a", a)
    .tag("icao_b", b)
    .field("tca_s", float(frame.get("tca_s", 0.0)))
    .field("min_sep_m", float(frame.get("min_sep_m", 0.0)))
    .field("controller_ts", float(frame.get("ts", 0.0)))
    .time(time.time_ns()))
alt_d = frame.get("alt_diff_ft")
if alt_d is not None:
    p = p.field("alt_diff_ft", int(alt_d))
```

7.2 Measurement `skywatch_status` (heartbeat)

One Point every 10 s plus one on startup so the dashboard always has fresh data even when nothing is colliding.

Tag	Value
source	gui
label	startup or tick

Field	Type	Notes
alive	int	always 1

8. In-memory aircraft record (controller side)

For completeness — this is what the controller stores per aircraft after fusing all the inputs above. Useful for the draw.io diagram if you want to show what each block holds.

Source: `controller/src/common/src/aircraft_t.h`.

```

#define AIRCRAFT_ICAO_LEN      6 /* 6 hex chars; storage = LEN+1 with NUL */
#define AIRCRAFT_ICAO_BUF_LEN (AIRCRAFT_ICAO_LEN + 1)

#define AIRCRAFT_VALID_ALT      (1u << 0)
#define AIRCRAFT_VALID_VEL      (1u << 1)
#define AIRCRAFT_VALID_HDG      (1u << 2)
#define AIRCRAFT_VALID_PRED      (1u << 3) /* Kalman pred_lat/lon valid */

enum aircraft_source {
    SRC_UNKNOWN = 0,
    SRC_ADS_B   = 1,
    SRC_BLE_SIM = 2,
    SRC_FICT     = 3,
};

struct aircraft_t {
    char    icao[AIRCRAFT_ICAO_BUF_LEN]; /* lowercase hex, NUL-terminated */
    double  lat; /* WGS-84 degrees */
    double  lon; /* WGS-84 degrees */
    double  ts; /* POSIX epoch seconds (float for sub-second) */
    enum aircraft_source source;
    uint8_t valid_mask; /* AIRCRAFT_VALID_* */
    int32_t alt_ft; /* valid only if AIRCRAFT_VALID_ALT */
    double  vel_kt; /* valid only if AIRCRAFT_VALID_VEL */
    double  hdg_deg; /* valid only if AIRCRAFT_VALID_HDG; 0..360 */
    double  pred_lat; /* Kalman 10s-ahead lat – valid if AIRCRAFT_VALID_PRED */
    double  pred_lon; /* Kalman 10s-ahead lon – valid if AIRCRAFT_VALID_PRED */
};

```

Fusion policy: **freshhest observation wins**. Both BLE_SIM and ADS_B feeds compete for the same row (keyed by ICAO); upsert overwrites every field and refreshes `last_seen_ms`. There is no per-field blending or weighted average.

Stale eviction: entries not refreshed within 10 s are removed by a background reaper thread (so a dropped aircraft fades from the GUI rather than ghosting forever).

9. Summary — every link at a glance

#	Link	Direction	Transport	Framing	Update rate	Struct / format
1	BLE aircraft frame	Sim → Controller	GATT NOTIFY	38-byte binary LE	5 Hz	<code>ble_aircraft_frame</code>
2	BLE warning frame	Controller → Sim	GATT WRITE (no resp)	16-byte binary + XOR CRC	On transition + ~8 s re-fire	<code>ble_warning_frame</code>
3a	JSON aircraft (ingress)	Host → Controller	USB-CDC ttyACM1	Newline-delimited JSON	2 Hz (dump1090)	<code>AircraftFrame</code>
3b	JSON aircraft (egress)	Controller → Host	USB-CDC ttyACM1	Newline-delimited JSON	5 Hz collision tick + per-update	<code>AircraftFrame</code>
3c	JSON collision	Controller → Host	USB-CDC ttyACM1	Newline-delimited JSON	On transition + while non-CLEAR	<code>CollisionFrame</code>
4	Controller shell	Host ↔ Controller	USB-CDC ttyACM0	Newline text (Zephyr shell)	Interactive	<code>skywatch <cmd></code>
5a	Sim position JSON	Sim → Host	USB-CDC ttyACM0	Newline-delimited JSON	5 Hz	<code>{lat,lon,alt,spd,hdg}</code> (int32×1e7 + metres + knots + deg)
5b	Sim WARN text	Sim → Host	USB-CDC ttyACM0	Newline text	On BLE write	<code>WARN <type> [args]</code>
5c	Sim shell	Host → Sim	USB-CDC ttyACM0	Newline text (Zephyr shell)	Interactive	<code>aircraft <cmd></code>
6a	MQTT collision (broadcast)	Controller host → Broker	TCP MQTT 3.1.1	retained, QoS 0	Per collision frame	<code>skywatch/collision ⇒ CollisionFrame JSON</code>
6b	MQTT collision (per-pair)	Controller host → Broker	TCP MQTT 3.1.1	retained, QoS 0	Per collision frame	<code>skywatch/collision/<a>_ ⇒ CollisionFrame JSON</code>
6c	MQTT status	Controller host → Broker	TCP MQTT 3.1.1	retained, QoS 0 + LWT	On connect / disconnect	<code>skywatch/status ⇒ "online" / "offline"</code>
7a	InfluxDB events	Controller host → Cloud	HTTPS REST	Line protocol	Per collision frame	measurement <code>skywatch_events</code>
7b	InfluxDB heartbeat	Controller host → Cloud	HTTPS REST	Line protocol	10 s	measurement <code>skywatch_status</code>

10. Where each spec lives in the repo

Document / link	File
BLE aircraft frame	<code>sim/mobile/src/common/src/ble_packet.h</code> + <code>controller/src/common/src/ble_packet.h</code>
BLE warning frame	<code>controller/src/common/src/ble_packet.h</code>
JSON wire format (Python)	<code>resources/standards/json_protocol.py</code>
JSON wire format (firmware)	<code>controller/src/json/src/json_parser.c</code> (zephyr descriptor mirror)
Collision detector	<code>controller/src/collision/src/collision.{c,h}</code>
ADS-B normalisation	<code>resources/standards/json_protocol.py</code> (<code>normalize_dump1090</code>) + <code>sdr_bridge.py</code>
Sim position + WARN	<code>sim/mobile/src/main/src/codein.c</code>
MQTT publisher	<code>controller/host/mqtt_publisher.py</code>
InfluxDB writer	<code>controller/host/influx_writer.py</code>
Aircraft record (in-memory)	<code>controller/src/common/src/aircraft_t.h</code>

If you need to redraw the protocol diagram, this section's table is the index — every wire format lives in exactly one of those files.